Enrolment No.:

Total No. of Questions - 10] [Total No. of Printed Pages - 2

BTCSSEM7 / SEVENTH SEMESTER
21BTCS030 / HIGH PERFORMANCE COMPUTING
(2021 PATTERN)

Time : 2:30 **Maximum Marks : 60**

N.B. :-

(i) Neat diagram must be drawn wherever necessary.
(ii) Figure to the right indicates full marks.
(iii) Assume suitable data wherever necessary.
(iv) Use of electronic pocket calculator is allowed (non-programmable).

Q1. A) Consider the following code for 3 iterations **[6] CO1 [L3]**
Load R1,[R2]
Load R3,[R4]

Assume cache-hit ratio is 75% ,for the 1st iteration 100ns time is required to fetch the data from memory. If 1ns is required for cache access. Calculate the average memory access time.

B) Explain the challenges of maintaining data consistency in a shared memory system versus a distributed memory system. How do synchronization mechanisms differ between the two architectures? **[6] CO1 [L4]**

(OR)

Q2. A) Analyze the differences in architecture between GPUs and CPUs. Discuss why GPUs are better suited for data-parallel tasks, providing a specific scenario to illustrate your analysis. **[6] CO1 [L4]**

B) Analyze the trade-offs between memory transfer overhead and computational speedup in GPU programming. **[6] CO1 [L5]**

Q3. A) Consider a scenario where you have to parallelize a program that performs matrix multiplication using OpenMP. Your task is to implement parallelization using both static and dynamic scheduling, and compare the execution time of each approach. **[6] CO2 [L4]**

Note* :

- Implement a serial version of matrix multiplication in C/C++.
- Parallelize the matrix multiplication using OpenMP with static scheduling.
- Parallelize the matrix multiplication using OpenMP with dynamic scheduling.
- Measure the execution time of each parallelized version for various matrix sizes.
- Compare the execution times and discuss the advantages and disadvantages of static and dynamic scheduling in this context.

B) Write an OpenMP program to perform parallel QuickSort on an array of integers. Use OpenMP directives to parallelize the partitioning step of the QuickSort algorithm. **[6] CO2 [L5]**

Questions:

- **Q1:** What part of the QuickSort algorithm can be parallelized using OpenMP, and why is the partitioning step a good candidate for parallelization?
- **Q2:** How would you handle thread synchronization during the partitioning step in the parallel QuickSort algorithm to avoid race conditions?
- **Q3:** Would using OpenMP for parallel QuickSort result in a performance improvement for small arrays or large arrays? Explain your reasoning.

(OR)

Q4. A) What is the purpose of the reduction clause in OpenMP? Write a program that calculates the sum of an array using the reduction clause. **[6] CO2 [L3]**

B) What is loop scheduling in OpenMP? Write a program that uses static scheduling to distribute iterations of a loop among threads. **[6] CO2 [L4]**

Q5. A) Write a MPI program simulating a bus topology with 4 processes. **[6] CO3 [L3]**

B) Write a MPI program to find Factorial of a Number. **[6] CO3 [L4]**

(OR)

Q6. A) What are the differences between collective communication and point-to-point communication? Provide examples of MPI_Barrier and MPI_Bcast to demonstrate collective communication. **[6] CO3 [L4]**

B) Implement a parallel prefix sum (scan) operation using MPI. Each process should compute the prefix sum of its portion of an array, and the results should be combined to compute the global prefix sum using MPI_Scan. **[6] CO3 [L5]**

Q7. A) Explain how the grid and block dimensions are specified and how they relate to the total number of threads. **[6] CO4 [L4]**

B) Write a CUDA C program that: **[6] CO5 [L5]**

1. Prints "This is the Fibonacci sequence calculation from host" from the host.
2. Launches a kernel with 10 threads. Each thread should calculate and print the Fibonacci number for its thread index. The thread with index 0 should calculate Fibonacci(0), thread 1 should calculate Fibonacci(1), and so on up to Fibonacci(9).

Ensure that each thread prints its corresponding Fibonacci number in parallel.

(OR)

Q8. A) For a GPU architecture with 64 threads per warp and a maximum of 2048 threads per block, calculate the maximum number of warps per block. Write a CUDA kernel that utilizes the maximum number of threads per block to perform a simple vector operation. **[6] CO4 [L3]**

B) Given a grid configuration with **[6] CO4 [L4]**
GridDim.x=6,GridDim.y=2,GridDim.z=1, calculate the global block ID for the following blocks:
B(5,1,0),
B(3,0,0), and
B(2,1,0).

Q9. A) #include <stdio.h> **[6] CO5 [L3]**

```
int main() {
int A[8] = {1, 2, 3, 4, 5, 6, 7, 8};
int B[8] = {8, 7, 6, 5, 4, 3, 2, 1};
int dot_product = 0;

// Loop to compute dot product
for (int i = 0; i < 8; i++) {
dot_product += A[i] * B[i];
}

printf("Dot product: %d\n", dot_product);
return 0;
}
```

Perform loop unrolling for the above given code.

B) #include <stdio.h> **[6] CO5 [L4]**

```
#define SIZE 5
#define INCREMENT 10

int main() {
int i;
int arr[SIZE] = {1, 2, 3, 4, 5};
int total_sum = 0;

// Loop to compute the sum of elements incremented by a constant value
for (i = 0; i < SIZE; i++) {
total_sum += arr[i] + INCREMENT; }

printf("Total sum of elements incremented by constant: %d\n", total_sum);
return 0;
}
```

Perform loop invariant code motion in the above code.

(OR)

Q10. A) Why are trigonometric and logarithmic functions considered expensive operations? Suggest an optimization technique using lookup tables to avoid recomputing such functions repeatedly in a loop. Provide an example scenario. **[6] CO5 [L4]**

B) Given two algorithms for a problem: **[6] CO5 [L5]**
Algorithm A: $O(n^3)$ time complexity and $O(n^3)$ memory complexity.
Algorithm B: O(n log n) time complexity and O(n) memory complexity.
Discuss which algorithm would be more suitable for large-scale computations and why. How would constraints like parallelizability and prefactor costs influence your choice? Provide examples where the less efficient algorithm might still be preferable.